Project Presentation of CO-OP Project at Industry (Module-2) (22CS421)

On

On Demand Airflow Review Environments

Yatin, Yash Goyal , Vidhan Gupta, Anmol Singla

2210992569,2210992554,2210992523,2210931001

Supervised By

Mr. Rajat Takkar

Department of Computer Science and Engineering,

Chitkara University, Punjab

Introduction

- **The Core Issue:** In big data teams, staging environments persistently drift away from actual production setups, creating a major hurdle for developers trying to test their code.

- **Our Solution:** An automated system designed to spin up a temporary, dedicated Airflow space every time a developer opens a Merge Request.

- **The Impact:** By giving each feature branch its own exact replica of the main setup, developers can test their work accurately and independently. This makes getting new features into production much smoother and faster.

Problem Background

- **Staging Drift:** Developers currently push all tests and experiments into one single staging branch. Over time, this makes the staging setup look completely different from the actual production environment.

- **Deployment Bottlenecks:** When it is time to deploy, developers must hand-pick which code updates to push.

- **Risks:** Doing this manually is risky, often leads to mistakes, and breaks good version control habits. This results in delayed release cycles

Proposed Solution

- **Review Spaces:** An automated system that sets up separate review spaces for building Airflow DAGS.
- **Pre-Merge Checking:** Allows the team to double-check their work before merging anything into the production environment.
- **CI/CD Integration:** The entire process kicks off automatically through GitLab CI/CD pipelines.
- **Review Apps:** It uses Kubernetes to manage container orchestration and Helm to handle deployment details, creating "Review Apps"

Technology Stack

- **Automation:** GitLab CI/CD runs the automation.
- **Container Orchestration:** Kubernetes (specifically EKS) hosts the containers.
- **Configuration Management:** Helm manages the complex configuration and deployment details.
- **Containerization:** Everything is packed into Docker containers.
- **Database:** PostgreSQL is used to store the metadata.

Implementation Steps

The rollout is planned in three distinct steps:

1. Getting the basic CI/CD working.
2. Managing the full setup and teardown process.
3. Tweaking everything for better performance and security.

Outcomes & Benefits

- **Independent Playgrounds:** Delivers a hands-off system giving every MR its own Airflow playground.
- **Faster Coding:** Speeds up development because developers no longer have to wait in line to test their work.
- **Fewer Bugs:** Cuts down on bugs since everything gets tested in total isolation.
- **Efficiency:** Stops the staging branch from getting cluttered, reduces duplicating DAGs, and saves money by destroying cloud resources immediately after testing.

Conclusion

- **Complete Automation:** Delivers a complete, automated way to handle short-lived Airflow setups.

- **Solving Staging Drift:** Successfully connects GitLab Review Apps with Kubernetes clusters to solve the persistent problem of staging environments drifting away from production in big data teams.

Thank You